

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**
称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #01: Relational Model & Relational Algebra

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Databases

A *database* is an organized collection of inter-related data that models some aspect of the real-world (e.g., modeling the students in a class or a digital music store). People often confuse “databases” with “database management systems” (e.g., MySQL, Oracle, MongoDB). A database management system (DBMS) is the software that manages a database.

Consider a database that models a digital music store (e.g., Spotify). Let the database hold information about the artists and which albums those artists have released.

2 Flat File Strawman

Database is stored as comma-separated value (CSV) files that the DBMS manages. Each entity will be stored in its own file. The application has to parse files each time it wants to read or update records. Each entity has its own set of attributes, so in each file, different records are delimited by new lines, while each of the corresponding attributes within a record are delimited by a comma.

Keeping along with the digital music store example, there would be two files: one for artist and the other for album. An artist could have a name, year, and country attributes, while an album has name, artist and year attributes.

Issues with Flat File

- **Data Integrity**
 - How do we ensure that the artist is the same for each album entry?
 - What if somebody overwrites the album year with an invalid string?
 - How do we store that there are multiple artists on one album?
- **Implementation**
 - How do you find a particular record?
 - What if we now want to create a new application that uses the same database?
 - What if two threads try to write to the same file at the same time?
- **Durability**
 - What if the machine crashes while our program is updating a record?
 - What if we want to replicate the database on multiple machines for high availability?

3 Database Management System

A *DBMS* is software that allows applications to store and analyze information in a database.

A general-purpose DBMS is designed to allow the definition, creation, querying, updating, and administration of databases.

Early DBMSs

Database applications were difficult to build and maintain because there was a tight coupling between logical and physical layers. The logical layer is which entities and attributes the database has while the physical layer is how those entities and attributes are being stored. Early on, the physical layer was defined in the application code, so if we wanted to change the physical layer the application was using, we would have to change all of the code to match the new physical layer.

4 Relational Model

Ted Codd noticed that people were rewriting DBMSs every time they wanted to change the physical layer, so in 1970 he proposed the relational model to avoid this. This relational model has three key points:

- Store database in simple data structures (relations).
- Access data through high-level language.
- Physical storage left up to implementation.

A *data model* is a collection of concepts for describing the data in a database. The relational model is an example of a data model.

A *schema* is a description of a particular collection of data, using a given data model.

The relational data model defines three concepts:

- **Structure:** The definition of relations and their contents. This is the attributes the relations have and the values that those attributes can hold.
- **Integrity:** Ensure the database's contents satisfy constraints. An example constraint would be that any value for the year attribute has to be a number.
- **Manipulation:** How to access and modify a database's contents.

A *relation* is an unordered set that contains the relationship of attributes that represent entities. Since the relationships are unordered, the DBMS can store them in any way it wants, allowing for optimization.

A *tuple* is a set of attribute values (also known as its *domain*) in the relation. Originally, values had to be atomic or scalar, but now values can also be lists or nested data structures. Every attribute can be a special value, NULL, which means for a given tuple the attribute is undefined.

A relation with n attributes is called an *n -ary relation*.

Keys

A relation's *primary key* uniquely identifies a single tuple. Some DBMSs automatically create an internal primary key if you do not define one. A lot of DBMSs have support for autogenerated keys so an application does not have to manually increment the keys.

A *foreign key* specifies that an attribute from one relation has to map to a tuple in another relation.

5 Data Manipulation Languages (DMLs)

A language to store and retrieve information from a database. There are two classes of languages for this:

- **Procedural:** The query specifies the (high-level) strategy the DBMS should use to find the desired result.
- **Non-Procedural:** The query specifies only what data is wanted and not how to find it.

6 Relational Algebra

Relational Algebra is a set of fundamental operations to retrieve and manipulate tuples in a relation. Each operator takes in one or more relations as inputs, and outputs a new relation. To write queries we can “chain” these operators together to create more complex operations.

Select

Select takes in a relation and outputs a subset of the tuples from that relation that satisfy a selection predicate. The predicate acts like a filter, and we can combine multiple predicates using conjunctions and disjunctions.

Syntax: $\sigma_{\text{predicate}}(R)$.

Projection

Projection takes in a relation and outputs a relation with tuples that contain only specified attributes. You can rearrange the ordering of the attributes in the input relation as well as manipulate the values.

Syntax: $\pi_{A_1, A_2, \dots, A_n}(R)$.

Union

Union takes in two relations and outputs a relation that contains all tuples that appear in at least one of the input relations. Note: The two input relations have to have the exact same attributes.

Syntax: $(R \cup S)$.

Intersection

Intersection takes in two relations and outputs a relation that contains all tuples that appear both of the input relations. Note: The two input relations have to have the exact same attributes.

Syntax: $(R \cap S)$.

Difference

Difference takes in two relations and outputs a relation that contains all tuples that appear in the first relation but not the second relation. Note: The two input relations have to have the exact same attributes.

Syntax: $(R - S)$.

Product

Product takes in two relations and outputs a relation that contains all possible combinations for tuples from the input relations.

Syntax: $(R \times S)$.

Join

Join takes in two relations and outputs a relation that contains all the tuples that are a combination of two tuples where for each attribute that the two relations share, the values for that attribute of both tuples is the same.

Syntax: $(R \bowtie S)$.

Observation

Relational algebra is a procedural language because it defines the high level-steps of how to compute a query. For example, $\sigma_{b_id=102}(R \bowtie S)$ is saying to first do the join of R and S and then do the select, whereas $(R \bowtie (\sigma_{b_id=102}(S)))$ will do the select on S first, and then do the join. These two statements will actually produce the same answer, but if there is only 1 tuple in S with b_id=102 out of a billion tuples, then $(R \bowtie (\sigma_{b_id=102}(S)))$ will be significantly faster than $\sigma_{b_id=102}(R \bowtie S)$.

A better approach is to say the result you want, and let the DBMS decide the steps it wants to take to compute the query. SQL will do exactly this, and it is the de facto standard for writing queries on relational model databases.

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**称为 **AI 内容创作者**？回复 [添砖加瓦]